

Parcial 2 - Programación 2

Tema: Food Store - Implementación del Modelo UML en Java (Versión Reducida)

Modalidad: Sin persistencia JDBC - Solo POO en memoria.

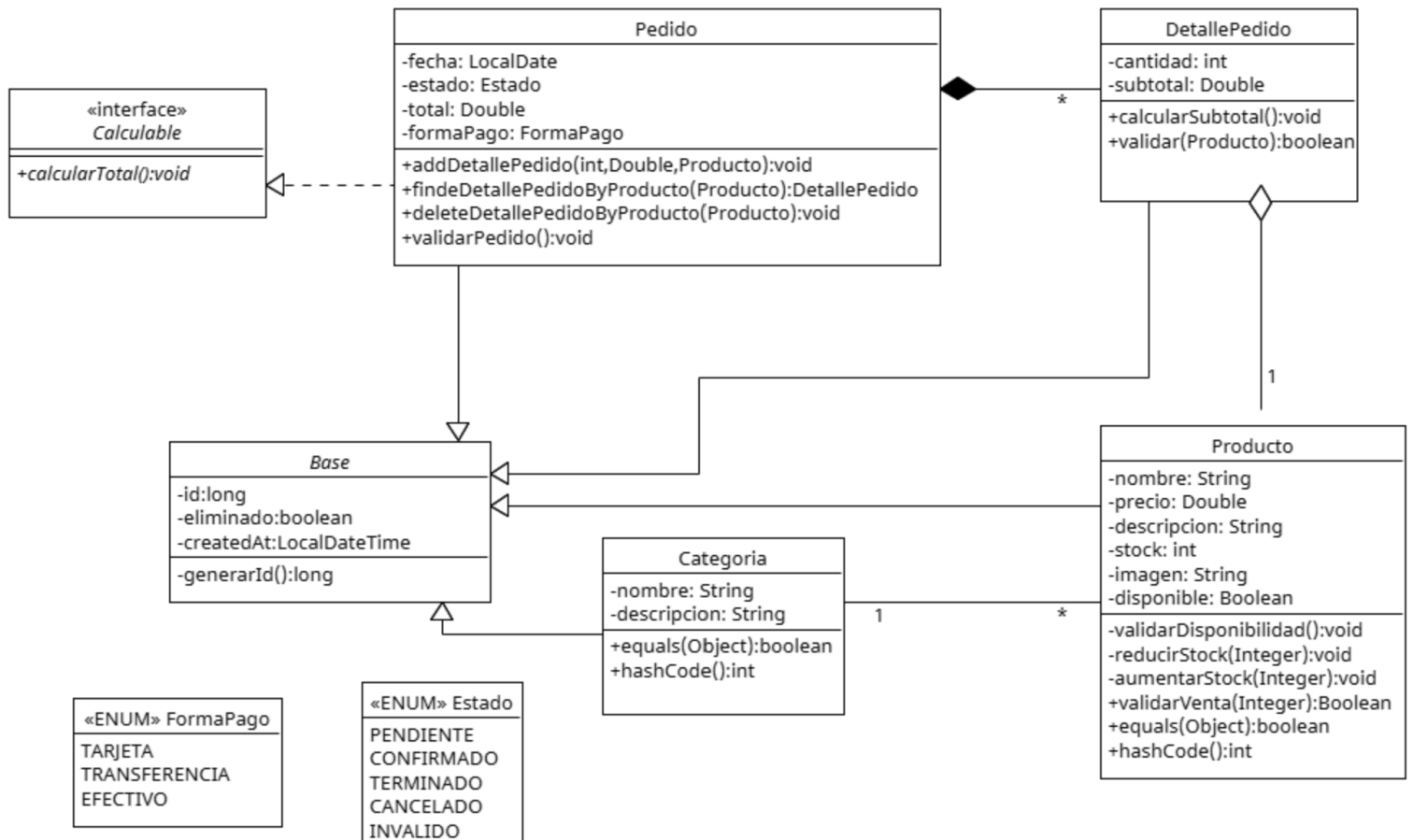
1. Objetivo

Implementar en Java el modelo orientado a objetos del sistema *Food Store* a partir del diagrama UML provisto por la cátedra. El foco de esta evaluación es verificar que el alumno domina los conceptos fundamentales de la Programación Orientada a Objetos (POO): herencia, interfaces, encapsulamiento, polimorfismo y, muy especialmente, la correcta lectura e implementación de las relaciones entre clases (Asociación, Agregación y Composición) deduciéndolas exclusivamente del diagrama UML.

2. Consigna General

A partir del diagrama UML entregado, el alumno deberá:

1. Crear un proyecto Java (Java 11 o superior).
 2. Implementar TODAS las clases, interfaces y enums definidos en el UML.
 3. Respetar estrictamente la jerarquía de herencia y los contratos de las interfaces.
 4. Deducir e implementar las relaciones: La cátedra no especificará si un objeto debe recibirse por constructor, inyectarse por un setter o construirse internamente (con `new`). Es responsabilidad del alumno observar los tipos de flechas y rombos en el UML para programar correctamente el ciclo de vida de cada relación.
 5. Desarrollar la clase Main: El alumno deberá construir su propio script de ejecución dentro del método `main()`, siguiendo estrictamente los casos de prueba y el orden detallado en la Sección 4 de este documento para demostrar la robustez de su código.
-



3. Reglas de Negocio y Lógica Interna

Para complementar el diagrama UML, se deben respetar las siguientes reglas de negocio durante la codificación:

Generales:

- Presten especial atención a la multiplicidad y a los tipos de rombos (vacíos o llenos) en el UML para decidir la forma correcta de implementar las relaciones de cardinalidad 1 a N y manejar sus listas.
- Se permite y recomienda agregar métodos *Getters* y *Setters* según sea necesario para mantener el encapsulamiento, así como sobrescribir el método `toString()` en las clases que lo requieran para una salida prolija.

Clase `Base` (Abstracta):

- El atributo `id` debe ser autogenerado secuencialmente por el sistema (se recomienda el uso de un contador estático). No debe solicitarse por parámetro en los constructores.
- El atributo `eliminado` debe nacer siempre en `false`.
- El atributo `createAt` debe tomar la fecha y hora del instante de su creación utilizando `LocalDateTime.now()`.
- Los constructores de las clases hijas no deben recibir estos atributos.

Clase `Producto`:

- El atributo `disponible` es de solo lectura y calculado. No se recibe por parámetro. Un producto nace disponible si su stock inicial es mayor a 0.
- El método `validarVenta(Integer cantidad)` debe comprobar si existe stock suficiente para realizar la venta. En caso de no existir stock suficiente, deberá lanzar una excepción adecuada indicando el motivo del error. Si la validación es exitosa, deberá invocar a un comportamiento interno que reduzca el stock.
- Si al reducir el stock este llega a 0, el atributo `disponible` debe pasar automáticamente a `false` y mostrarse un mensaje de alerta por consola indicando que el producto se agotó.

Clase `DetallePedido`:

- Los atributos `subtotal` son de solo lectura y calculados. No se reciben por parámetro.

- Al construirse, el detalle debe validar si la venta es posible (consultando al producto). Si la operación no es válida (por falta de stock, producto nulo o cualquier otra inconsistencia), deberá propagarse o capturarse la excepción correspondiente según el diseño elegido por el alumno.
- El `subtotal` debe auto-calcularse multiplicando la cantidad solicitada por el precio del producto.

Clase Pedido:

- El atributo `fecha` debe autogenerarse tomando la fecha actual (`LocalDate.now()`).
- El atributo `estado` no se recibe por constructor. Al nacer, como el pedido no tiene detalles, su estado inicial debe ser `INVALIDO`.
- Al agregar un `DetallePedido` válido de forma exitosa, el estado del pedido debe pasar automáticamente a `PENDIENTE`.
- Cálculos y total: El método `calcularTotal()` (proveniente de la interfaz `Calculable`) debe sumar los subtotales de los detalles válidos. Este recálculo debe ejecutarse automáticamente cada vez que se modifique la lista de detalles (al agregar o al eliminar).

4. Clase Main (Script de Pruebas)

El alumno deberá construir el método `main` escribiendo el código necesario para ejecutar, en orden, los siguientes 5 pasos. Cada paso debe imprimir subtítulos o separadores por consola para identificar qué se está probando:

- **Paso 1: Creación del Catálogo**
 - Instanciar 2 Categorías (ej. "Lácteos" y "Panadería").
 - Instanciar 4 Productos asociándolos a dichas categorías.
 - *Condición estricta:* Al menos uno de los productos debe crearse con un stock muy bajo (ej. 2 unidades). Imprimir los productos creados.
- **Paso 2: Creación de Pedido Inicial**
 - Instanciar un `Pedido` (pasando únicamente la forma de pago, ej. `EFFECTIVO`).
 - Imprimir el pedido recién creado. Debe evidenciarse que nace con un Total de \$0.0 y en Estado `INVALIDO`.
- **Paso 3: Prueba de Venta Fallida (Exceso de Stock)**
 - Intentar agregar al pedido un detalle (`addDetallePedido`) solicitando una cantidad mayor al stock existente de alguno de los productos.
 - Comprobación: La operación deberá generar una excepción debido al exceso de stock solicitado. El alumno deberá demostrar su correcta captura y manejo mediante una estructura `try-catch`, mostrando el mensaje asociado a la excepción.

- **Paso 4: Prueba de Venta Exitosa y Agotamiento de Stock**
 - Intentar agregar al pedido un detalle (`addDetallePedido`) solicitando exactamente la misma cantidad de stock que le queda al producto de "stock muy bajo" creado en el Paso 1 (ej. pedir 2 unidades del producto que tiene 2).
 - *Comprobación:* La consola debe imprimir la alerta interna del Producto indicando que se ha quedado sin stock.
 - Imprimir el producto: Su disponibilidad debe haber cambiado a `false` y su stock a 0.
 - Imprimir el pedido: Su estado debe haber cambiado a `PENDIENTE`, debe contener 1 ítem en su lista y el Total debe haberse calculado correctamente.
- **Paso 5: Impresión final**
 - Ejecutar un `System.out.println(pedido)` final para mostrar el estado íntegro y consolidado del objeto Pedido con todas sus relaciones anidadas, demostrando el correcto funcionamiento del polimorfismo mediante los `toString()` sobrescritos.

5. Entregables y Restricciones

- **Código Fuente:** Al finalizar, el alumno debe entregar un archivo `.zip` con la carpeta del proyecto Java completo.
- **Compilación:** El proyecto debe compilar y ejecutarse sin errores utilizando la clase `Main` proporcionada. No se aceptan entregas con errores de compilación.
- **Nomenclatura:** El archivo debe nombrarse estrictamente `Apellido_Nombre_ExamenProg2.zip`.
- **Restricciones:** * Está PROHIBIDO el uso de librerías externas que no sean las del JDK estándar.
- No se permite el uso de JDBC, JPA, Hibernate ni ninguna librería de persistencia en base de datos.
- No se requiere implementar menús interactivos por consola ni el patrón DAO. El foco es la arquitectura de clases en memoria.